

ПОПРАВКА С ДОПЪЛНЕНИЕ ВЪРХУ ПРАКТИЧЕСКИ ИЗПИТ ЗА МЕТОДИ

Имаше ученици с решения със списъци и речници – тях не ги показвам, т.к. са извън учебния план.

Ако имаме променлива с плаваща запетая тип `double` или `decimal`, какво форматиране да предпочетем за извеждане на резултата, например: `{variable:C2}`

Формат	Пример	Описание	Употреба
:C2	123.45 лв.	Паричен формат	Пари, валути
:D2	05	Цяло число с водещи нули	ID, кодове, дати
:00.00	12.30	Децимален формат	Измервания, проценти
:##.##	12.3	Компактен децимален	Научни данни

Число	C2	D2	00.00	##.##
0	0,00 лв.	00	00.00	
5	5,00 лв.	05	05.00	5
5.5	5,50 лв.	05	05.50	5.5
12.3	12,30 лв.	12	12.30	12.3
123.456	123,46 лв.	123	123.46	123.46
1234.56	1 234,56 лв.	N/A	1234.56	1234.56
12345.67	12 345,67 лв.	N/A	12345.67	12345.67

:F2 = Format 2 – форматиране, закръгляне с точност 0,5 до 20-я знак след запетаята, еквивалентно на `Math.Round(variable)` или явно изписано `Math.Round(variable, 2)`.

Лесни формати за подравняване:

:C2 = Currency 2 – **за пари** - добавяне на 0-и след запетаята, например .3 в 2 позиции става .03. Използва се за валути - \$, лв, Еу и добавя автоматично разделител за хиляди 1234.5 -> 1 234.50

:D2 = Decimal 2 – **за време, ID, кодове** - когато има число като 3, да се добавят 0-и докато стане 03. Работи само с цели числа `int`, `long`, но не поддържа десетични `double`, `decimal`.

В науката, за по-голяма точност се използват още 2 формата:

00.00 – добавя нули на първи и последни позиции като едновременно C2 и D2, например 5.7 -> 05.70. Полезен е, когато искаме да фиксираме дължина, `id = 5 -> {id:00000} -> 00005`

##.## - прави обратното – по аналогия на Excel свитите клетки с числа – премахва излишните 0-и от предни и последни позиции, например 03.80 -> 3.8 или 0.500 -> .5 или 0.567 -> 5.67 или 0 -> „“

Формат	Предимства	Недостатъци	Употреба
00.00	 Фиксиран формат  Подравняване  Винаги показва цифри	 Заема повече място  Излишни нули	Пари, време, ID
##.##	 Компактен  Чисто визуално  Скрива нули	 Празни стрингове  Неподравнен	Проценти, научни данни, UI

Ако имате само 2-3 случая или сложни условия – тогава изберете if-else, иначе switch, който работи директно с hash-таблица.

Ресурс	If-Else	Switch
CPU време	Повече (последователни проверки)	 По-малко (директно изпълнение)
Памет	 Минимална	 Минимална (оптимизирана)
Четимост	 Трудна при много случаи	 Отлична
Поддръжка	 Сложна за промяна	 Лесна за разширяване

За конкретни стойности изберете SWITCH:

-  По-бърз (особено при 4+ случая)
-  По-ефикасен (по-малко CPU цикли)
-  По-четим код
-  По-лесен за поддръжка

```
switch(season)    // стар, класически метод, много лесен
{
    case 12:
    case 1:
    case 2: seasonName = "зима"; break;
    case 3:
    case 4:
    case 5: seasonName = "пролет"; break;
    case 6:
    case 7:
    case 8: seasonName = "лято"; break;
    case 9:
    case 10:
    case 11: seasonName = "есен"; break;
    default: monthName = "извън диапазона"; break;
```

```
}
```

Примерен синтаксис на **модерен switch** – имаме кръстосване в условието (holiday = “Нова година”), обикновени запетайки `,` за разделител и `;` чак в края на цялата конструкция.

```
string holiday = (day, month) switch
{
    (1, 1) => "Нова година!",
    (3, 3) => "Освобождение!",
    (1, 6) => "Ден на детето!",
    (15, 9) => "Начало на учебната година!",
    _ => "Обикновен ден."
};
```

При някои компютри, поради локални езикови настройки, има проблем с отпечатване на кирилица в терминала. Тогава добавяме:

```
Console.OutputEncoding = System.Text.Encoding.UTF8; // Кирилица
```

Отпечатване на конзолата:

1) **Стар синтаксис** с подредба по номера от 0-ва позиция и изреждане на променливите:

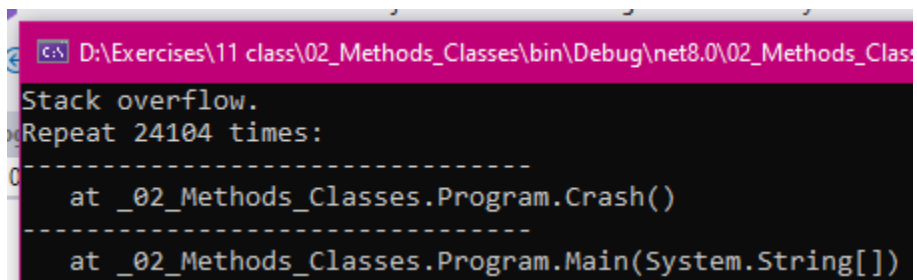
```
Console.WriteLine("Old syntax {0}, {1}, {2}, {3}", a, b, c, name);
```

2) **Нов синтаксис** – по-голяма четимост:

```
Console.WriteLine($"Newer syntax {a}, {b}, {c}, {name}");
```

И един любопитен въпрос: какво ще стане, ако даден метод извика сам себе си без да прави нищо:

```
static void Crash()
{
    Crash();
}
```



Това е рекурсия, която препълва стека заради повторения.

Да изясним още, че спазваме аналогични подредби и синтаксис: метод под метод, клас под клас. Main() е private-метод. Щом се намираме в метод, конвенцията е с малки букви. Методите, които са под Main(), са **static**, т.е. по-прости, защото са в същия клас и не се създава нов обект. Ако създадем друг клас, който съдържа сложна комбинация от прости типове като int, string и иска създаване на нов обект, **new** ще хвърли грешка със **static**.

```

0 references
static void Main(string[] args)
{
    Console.WriteLine($"
    Factorial of 6 = {Factorial(6)}
    Power of 2 ^ 3 = {Power(2,3)};");
    Crash();
}

2 references
int Factorial(int f)
{
    if (f <= 1) return 1;
    return f * Factorial(f - 1);
}

2 references
static int Power(int a, int b)
{
    if (b == 0) return 1;
    if (b == 1) return a;
    return a * Power(a, b - 1);
}

2 references
static void Crash()
{
    Crash();
}

```

метод

метод

метод

метод

static е по-прост,
няма обекти new

1 0

Ако не сме сигурни, просто отваряме и четем грешката – обектът изисква статичен метод, така че просто дописваме думата static.

Entire Solution	1 Error	0 of 10 Warnings	0 of 8 Messages	Build + IntelliSe
Code	Description			
CS0120	An object reference is required for the non-static field, method, or property 'Program.Factorial(int)'			

В задачата са показани рекурсии – методи, които извикват сами себе си и последният пример е най-интересен, т.к. програмата крашва заради препълване на стека от повторения.

```
D:\Exercises\11 class\01_exercise_27.10.25\bin\Debug\net8.0\01_exercise_27.10.25.exe

    Factorial of 6 = 720
    Power of 2 ^ 3 = 8;
Stack overflow. Crash() recursion
Repeat 24100 times:
-----
    at _01_exercise_27._10._25.Program.Crash()
-----
    at _01_exercise_27._10._25.Program.Main(System.String[])
```

```
static void Main(string[] args)
{
    Console.WriteLine($"@
    Factorial of 6 = {Factorial(6)}
    Power of 2 ^ 3 = {Power(2,3)};");
    Crash();
}

static int Factorial(int f)
{
    if (f <= 1) return 1;
    return f * Factorial(f - 1);
}
static int Power(int a, int b)
{
    if (b == 0) return 1;
    if (b == 1) return a;
    return a * Power(a, b - 1);
}
static void Crash()
{
    Crash();
}
```

Ето 3 възможни обръщания на текст:

```
public static void Chars()
{
    Console.Write("Enter text:");
    char[] chars = Console.ReadLine().ToCharArray();
    for (int i = chars.Length - 1; i >= 0; i--)
        Console.Write(chars[i]);
}
public static void Text()
{
    Console.Write("Enter text:");
    string input = Console.ReadLine(); // може на 1 ред
    char[] chars = input.ToCharArray(); // да се обединят двата реда
    Array.Reverse(chars); // Вграден метод
    string reversed = new string(chars);
    Console.WriteLine("Reversed text: " + reversed);
}
public static void Words()
```

```

{
    Console.WriteLine("Enter words");
    var words = Console.ReadLine().Split(' ').ToArray(); // Trim()
    for (int i = 0; i < words.Length / 2; i++)
    {
        var temp = words[i];
        words[i] = words[words.Length - 1 - i];
        words[words.Length - 1 - i] = temp;
    }

    Console.WriteLine(string.Join(" ", words));
}

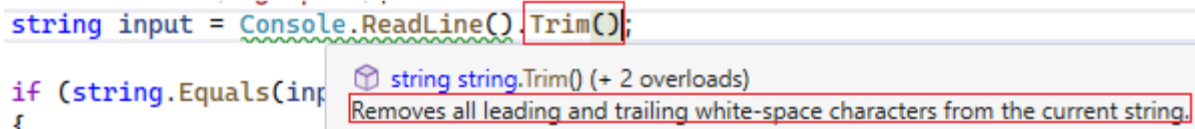
```

`.Trim()` – премахва интервали

`.Split(' ,')` – разделя, обикновено чрез символ, но може и чист `.Split(' ')`

`.Join(" ,")` – добавя разделител, също може `.Join(" ")`

`string input = Console.ReadLine().Trim().ToUpper();`



The screenshot shows the code `string input = Console.ReadLine().Trim();` with the `Trim()` method highlighted. A tooltip is displayed, showing the signature `string string.Trim() (+ 2 overloads)` and the description `Removes all leading and trailing white-space characters from the current string.`

Да си припомним While – подходящ, когато не знаем предварително броя или типа на въведените стойности. Ако имаме брояч `count++` го разполагаме последен, а преди него може да има вложени други операции като `sum +=` или в средата условия с `if()`- проверки. Зануляванията са преди цикъла, защото, ако са вътре в него, при всяка нова операция (завъртане), ще започва отначало.

```

string input = Console.ReadLine().ToLower();
int sum = 0, count = 0;

while(input != "stop")
{
    int n = int.Parse(input);
    if (n % 2 == 0 || n % 3 == 0)
    {
        Console.WriteLine(n + " ");
        sum += n;
        count++;
    }
    input = Console.ReadLine().ToLower();
}

Console.WriteLine((double)sum/count); //cast-ване изречно за (double) int / int да не губим стойност

```

Ето и едно примерно решение на Order от изпитването с do-while. Разликата е, че спестяваме едно четене от конзолата по този начин и всичко остава вътре в цикъла, освен декларирането с частно поле:

```

internal class Order
{
    public static decimal Choice()

```

```

{
    string input;          // private fields
    decimal sum = 0.0m;

    do
    {
        input = Console.ReadLine().ToLower();
        char order = char.Parse(input);

        sum = order switch
        {
            'a' => sum += 0.80m,
            'b' => sum += 1.70m,
            'c' => sum += 1.20m,
            'd' => sum += 1.50m,
            _ => sum += 0m,
        };
    } while (input != "bill");
    Total(sum);
    Converter(sum);
    return sum;
}
private static void Total(decimal totalSum)
{
    Console.WriteLine($"Total: {totalSum} BGN");
}
private static void Converter(decimal value)
{
    Console.WriteLine("1 EUR = 1.95583 BGN");
    decimal euro = value / 1.95583m;
    Console.WriteLine($"Converted currency: {euro}");
    Console.WriteLine($"Pay: {Math.Round(euro,2)} EUR");
}
}

```

Интересно решение с

```
if (string.Equals(input, "bill", StringComparison.OrdinalIgnoreCase))
```

Също, интересно решение имаше с goto:

```

        case "BILL": goto label;
        default:
            break;
    }

}

label:
    Console.WriteLine("Total: " + bill + " BGN");

```

...

Модификатори за достъп (Access Modifiers): Определят откъде може да бъде извикан методът.

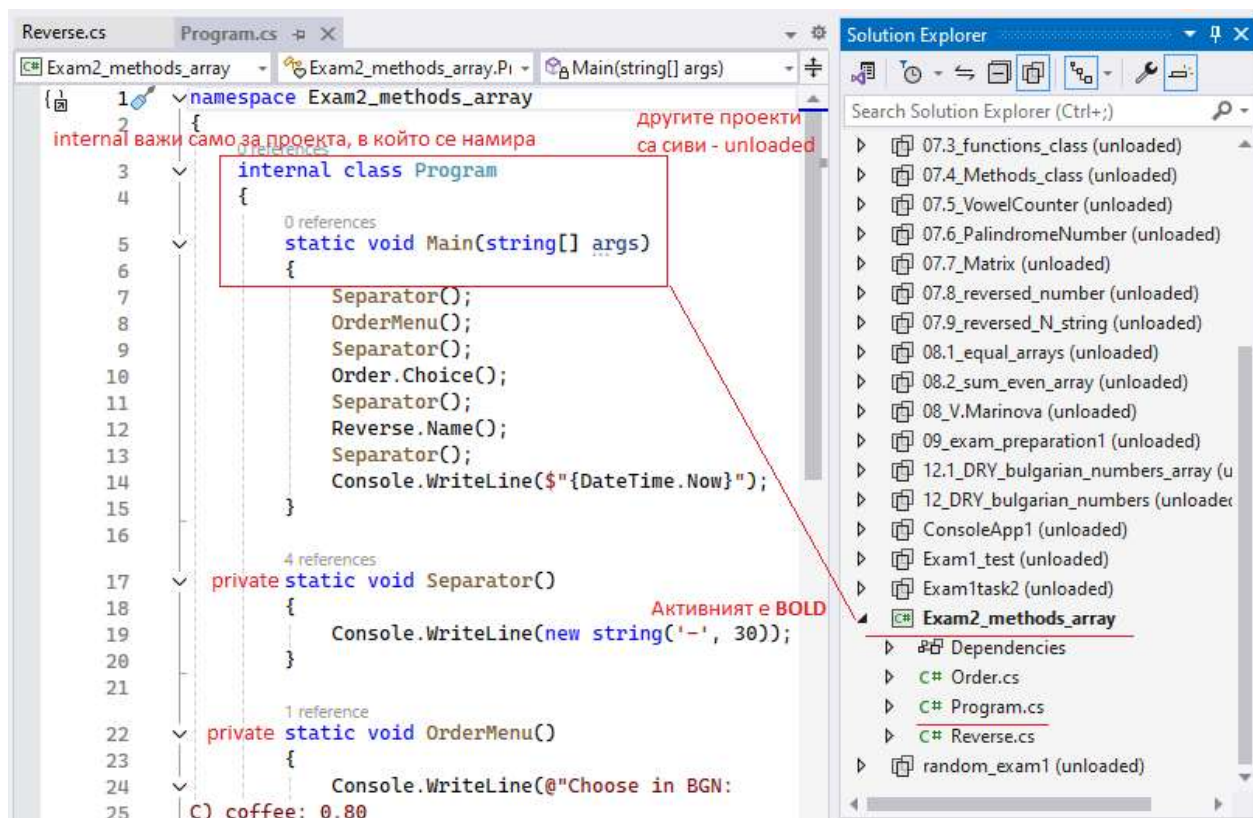
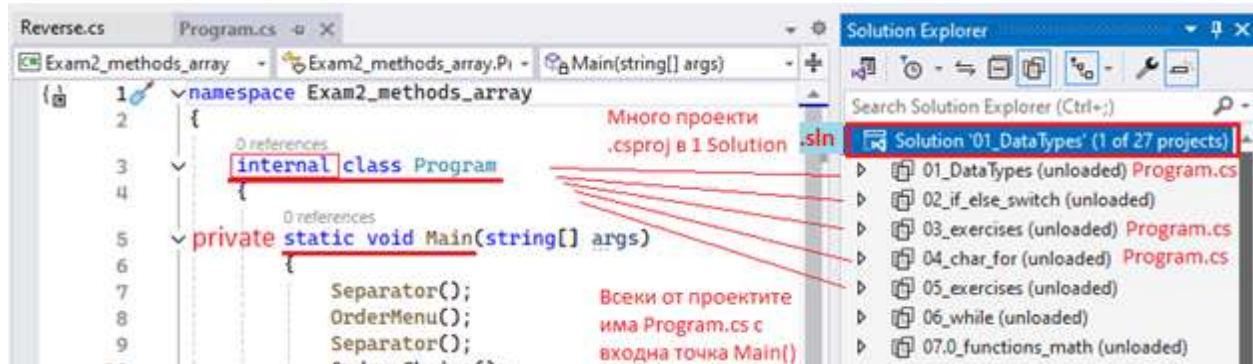
public: Методът е достъпен от всяко място. Той е най-често срещаният и подходящ за начинаещи, но не дава защита на данните.

private: Обикновено не го пише, защото е по подразбиране. Методът е достъпен само в рамките

на класа, в който е дефиниран. Използва се при влагане на 1 метод в друг за по-голяма защита. Вложеният метод е достъпен само за този, в чието тяло се извиква.

protected: Достъпен в рамките на класа и в наследяващи класове.

internal: Достъпен в рамките на текущия проект (assembly). Internal class Program – когато в един Solution имаме много проекти, се стартира само 1 Program (който държи Main()). Това е, защото internal го прави видим в рамките само на проекта, в който се намира, но не и в другите проекти. Ако си спомняте, давахме unload и reload на проектите за да ги деактивираме и активираме. Причината беше точно за да направим конкретен Program.cs видим.



Хвърля се грешка, когато в св вложим void метод, защото в един св влагаме още 1 св.
static


```

public class Rectangle
{
    public double Width; // properties
    public double Height; // променливи с главна буква, защото са public и са извън
    public double Area() // метод (могат да бъдат използвани от много методи)
    {
        return (Width * Height);
    }
}

public class Book ...

internal class Program
{
    static void Main(string[] args)
    {
        Rectangle abcd1 = new Rectangle(); // object
        abcd1.Width = 12.34; // обект. точка - наследяване (влизане) - свойство
        abcd1.Height = 56.78; // отстояние за по-добро подреждане по колони

        Console.WriteLine($"Width: {abcd1.Width, -20} Height: {abcd1.Height, -20} Area: {abcd1.Area():F2}");
        // на кой обект кое свойство // формат 2 знак след запетаята

        Rectangle abcd2 = new Rectangle();
        abcd2.Width = 9.87;
        abcd2.Height = 6.54;

        Console.WriteLine($"Width: {abcd1.Width, -20} Height: {abcd1.Height, -20} Area: {Math.Round(abcd2.Area(), 2)}");
        // вкарваме метод между скобите на друг метод // return, а не void !!!
    }
}

```

Понякога се случва, на пръв поглед, когато вложим даден метод в CW, да хвърля грешка. Причината е, че самият метод е тип void и самият той отпечатва CW. Така стават **2 CW които са вложени 1 в друг**. В такива случаи няма нужда от допълнителен CW, а направо извикваме метода (който отпечатва CW).